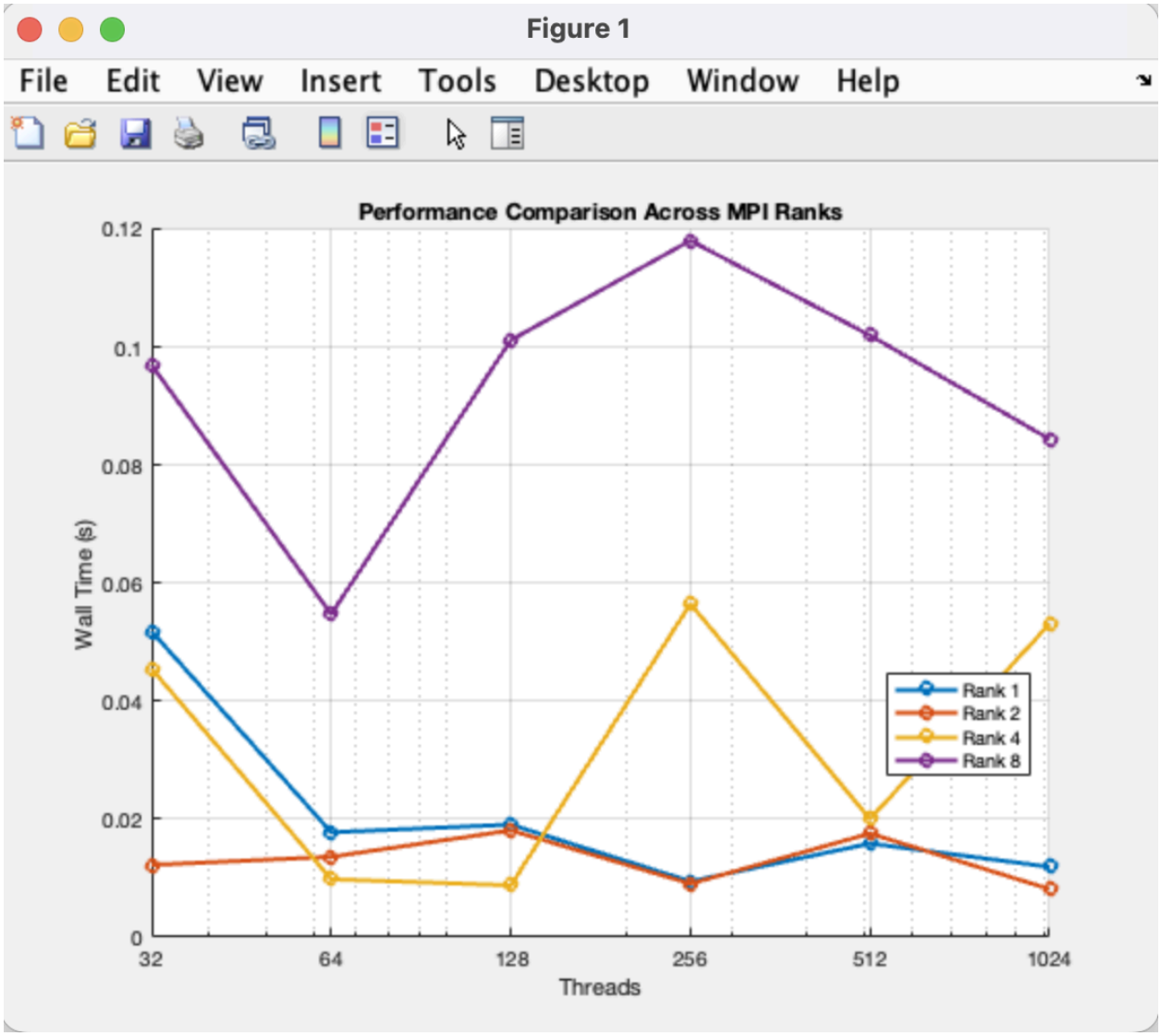
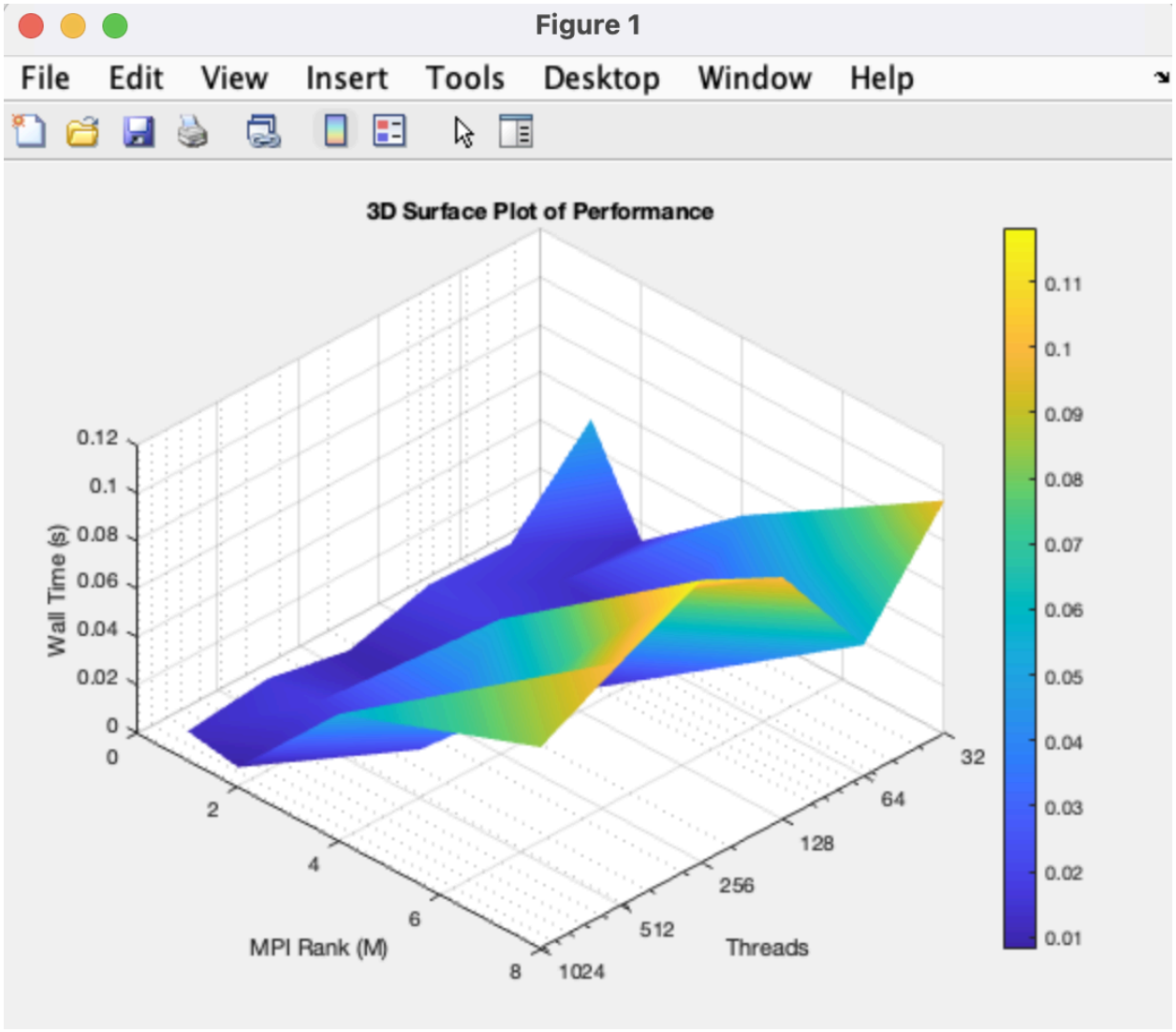
Brandon Nguyen, 827813045
COMPE596 - Spring 2026
5/4/2026

COMPE596 P11 - Hybrid MPI + CUDA Simpson's Quadrature Method

1) 3D Surface Plot





2) MATLAB Code

// 3D Surface Plot

```
clc;
clear;
close all;

T = readtable('p11_surface.csv');

threads_vals = unique(T.threads);
ranks_vals = unique(T.M);

[X, Y] = meshgrid(threads_vals, ranks_vals);
```

```
Z = zeros(length(ranks_vals), length(threads_vals));

for i = 1:length(ranks_vals)

    for j = 1:length(threads_vals)

        idx = (T.M == ranks_vals(i)) & ...
              (T.threads == threads_vals(j));

        Z(i,j) = T.wall_s(idx);
    end
end

figure;

surf(X, Y, Z);

shading interp;
colorbar;

xlabel('Threads');
ylabel('MPI Rank (M)');
zlabel('Wall Time (s)');

title('3D Surface Plot of Performance');

view(135, 30);

set(gca, 'XScale', 'log');

xticks([32 64 128 256 512 1024]);
xticklabels({'32','64','128','256','512','1024'});
```

// Comparison Plot

```
clc;
clear;
close all;

T = readtable('p11_surface.csv');
```

```
ranks = unique(T.M);

figure;
hold on;
grid on;

for i = 1:length(ranks)

    rank_val = ranks(i);

    idx = (T.M == rank_val);

    threads = T.threads(idx);
    walltime = T.wall_s(idx);

    [threads_sorted, order] = sort(threads);
    wall_sorted = walltime(order);

    plot(threads_sorted, wall_sorted, '-o', ...
        'LineWidth', 2, ...
        'DisplayName', sprintf('Rank %d', rank_val));
end

xlabel('Threads');
ylabel('Wall Time (s)');
title('Performance Comparison Across MPI Ranks');

legend('Location', 'best');

set(gca, 'XScale', 'log');

xticks([32 64 128 256 512 1024]);
xticklabels({'32','64','128','256','512','1024'});

hold off;
```

3) Code Listings

// Main Program

```
#define _USE_MATH_DEFINES
#include <cuda_runtime.h>
#include <cuda.h>
#include <math.h>
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,    \
                    cudaGetErrorString(err__));                             \
            MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);                        \
        }                                                                   \
    } while (0)

__device__ static double f_device(double x)
{
    double c = cos(x);
    double arg = c / (1.0 + 2.0 * c);
    if (arg > 1.0) {
        arg = 1.0;
    }
    if (arg < -1.0) {
        arg = -1.0;
    }
    return acos(arg);
}

__global__ static void simpson_panel_reduce_kernel(double a, double h,
                                                   unsigned long long panel_start,
                                                   unsigned long long panel_count,
                                                   double *block_sums)
{
    extern __shared__ double sdata[];
```

```
    unsigned int tid = threadIdx.x;
    unsigned long long global_panel =
        panel_start + (unsigned long long)blockIdx.x * blockDim.x + tid;

    double val = 0.0;

    if (global_panel < panel_start + panel_count) {
        double x0 = a + (double)(2ULL * global_panel) * h;
        double x1 = a + (double)(2ULL * global_panel + 1ULL) * h;
        double x2 = a + (double)(2ULL * global_panel + 2ULL) * h;
        val = f_device(x0) + 4.0 * f_device(x1) + f_device(x2);
    }

    sdata[tid] = val;
    __syncthreads();

    /* Tree reduction in shared memory; blockDim must be a power of two. */
    for (unsigned int stride = blockDim.x >> 1; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid == 0) {
        block_sums[blockIdx.x] = sdata[0];
    }
}

static int is_power_of_two(int x)
{
    return (x > 0) && ((x & (x - 1)) == 0);
}

static void parse_args(int argc, char **argv, unsigned long long *n_intervals,
                       int *threads_per_block, int *show_help)
{
    int i;
    *n_intervals = 10000000ULL;
```

```
    *threads_per_block = 256;
    *show_help = 0;

    for (i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-n") == 0 && (i + 1) < argc) {
            *n_intervals = strtoull(argv[++i], NULL, 10);
        } else if (strcmp(argv[i], "-t") == 0 && (i + 1) < argc) {
            *threads_per_block = atoi(argv[++i]);
        } else if (strcmp(argv[i], "-h") == 0 || strcmp(argv[i], "--help") == 0) {
            *show_help = 1;
        }
    }
}

int main(int argc, char **argv)
{
    const double a = 0.0;
    const double b = 0.5 * M_PI;
    const double exact = 5.0 * (M_PI * M_PI) / 24.0;

    int rank = 0;
    int size = 1;
    int device_count = 0;
    int device_id = 0;

    unsigned long long n_intervals = 0ULL;
    int threads_per_block = 0;
    int show_help = 0;

    unsigned long long total_panels = 0ULL;
    unsigned long long base_panels = 0ULL;
    unsigned long long rem_panels = 0ULL;
    unsigned long long local_panel_start = 0ULL;
    unsigned long long local_panel_count = 0ULL;

    double h = 0.0;
    int blocks = 0;
    double *d_block_sums = NULL;
    double *h_block_sums = NULL;
    double local_simpson_sum = 0.0;
```

```
double local_integral = 0.0;

double *all_partial = NULL;
double approx = 0.0;
double err_abs = 0.0;

float kernel_ms = 0.0f;
double local_kernel_s = 0.0;
double max_kernel_s = 0.0;
cudaEvent_t ev_start;
cudaEvent_t ev_stop;

double wall_start = 0.0;
double wall_end = 0.0;
double wall_elapsed = 0.0;

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

parse_args(argc, argv, &n_intervals, &threads_per_block, &show_help);
if (show_help) {
    if (rank == 0) {
        printf("Usage: ./p11 -n NINTERVALS -t THREADS_PER_BLOCK\n");
        printf("  -n : total Simpson subintervals (even)\n");
        printf("  -t : CUDA threads per block (power of 2)\n");
        printf("Example: mpirun -mca btl \"^openib\" -np 4 ./p11 -n 10000000 -t 256\n");
    }
    MPI_Finalize();
    return EXIT_SUCCESS;
}

if (n_intervals < 2ULL) {
    if (rank == 0) {
        fprintf(stderr, "Error: N must be >= 2\n");
    }
    MPI_Finalize();
    return EXIT_FAILURE;
}
if ((n_intervals & 1ULL) != 0ULL) {
```

```
        if (rank == 0) {
            fprintf(stderr, "Error: N must be even for Simpson 1/3\n");
        }
        MPI_Finalize();
        return EXIT_FAILURE;
    }
    if (!is_power_of_two(threads_per_block)) {
        if (rank == 0) {
            fprintf(stderr, "Error: -t must be a power of 2\n");
        }
        MPI_Finalize();
        return EXIT_FAILURE;
    }
    if (threads_per_block > 1024) {
        if (rank == 0) {
            fprintf(stderr, "Error: -t must be <= 1024\n");
        }
        MPI_Finalize();
        return EXIT_FAILURE;
    }

    CUDA_CHECK(cudaGetDeviceCount(&device_count));
    if (device_count <= 0) {
        if (rank == 0) {
            fprintf(stderr, "Error: no CUDA devices found\n");
        }
        MPI_Finalize();
        return EXIT_FAILURE;
    }

    device_id = rank % device_count;
    CUDA_CHECK(cudaSetDevice(device_id));

    total_panels = n_intervals / 2ULL;
    base_panels = total_panels / (unsigned long long)size;
    rem_panels = total_panels % (unsigned long long)size;

    local_panel_count = base_panels + ((unsigned long long)rank < rem_panels ? 1ULL :
0ULL);
    local_panel_start =
```

```
        (unsigned long long)rank * base_panels +
        ((unsigned long long)rank < rem_panels ? (unsigned long long)rank : rem_panels);

    h = (b - a) / (double)n_intervals;

    blocks = (int)((local_panel_count + (unsigned long long)threads_per_block - 1ULL) /
                   (unsigned long long)threads_per_block);

    if (blocks > 0) {
        h_block_sums = (double *)malloc((size_t)blocks * sizeof(double));
        if (h_block_sums == NULL) {
            fprintf(stderr, "Rank %d: host allocation failed\n", rank);
            MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
        }

        CUDA_CHECK(cudaMalloc((void **)&d_block_sums, (size_t)blocks *
sizeof(double)));
        CUDA_CHECK(cudaEventCreate(&ev_start));
        CUDA_CHECK(cudaEventCreate(&ev_stop));
    } else {
        CUDA_CHECK(cudaEventCreate(&ev_start));
        CUDA_CHECK(cudaEventCreate(&ev_stop));
    }

    MPI_Barrier(MPI_COMM_WORLD);
    wall_start = MPI_Wtime();

    CUDA_CHECK(cudaEventRecord(ev_start));
    if (blocks > 0) {
        simpson_panel_reduce_kernel<<<blocks, threads_per_block,
                                      (size_t)threads_per_block * sizeof(double)>>>(
            a, h, local_panel_start, local_panel_count, d_block_sums);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaMemcpy(h_block_sums, d_block_sums, (size_t)blocks *
sizeof(double),
                              cudaMemcpyDeviceToHost));
    }
    CUDA_CHECK(cudaEventRecord(ev_stop));
    CUDA_CHECK(cudaEventSynchronize(ev_stop));
    CUDA_CHECK(cudaEventElapsedTime(&kernel_ms, ev_start, ev_stop));
```

```
    local_kernel_s = 1.0e-3 * (double)kernel_ms;

    local_simpson_sum = 0.0;
    for (int i = 0; i < blocks; i++) {
        local_simpson_sum += h_block_sums[i];
    }
    local_integral = (h / 3.0) * local_simpson_sum;

    if (rank == 0) {
        all_partial = (double *)malloc((size_t)size * sizeof(double));
        if (all_partial == NULL) {
            fprintf(stderr, "Rank 0: host allocation failed\n");
            MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
        }
    }

    MPI_Gather(&local_integral, 1, MPI_DOUBLE, all_partial, 1, MPI_DOUBLE, 0,
MPI_COMM_WORLD);
    MPI_Reduce(&local_kernel_s, &max_kernel_s, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);

    wall_end = MPI_Wtime();
    wall_elapsed = wall_end - wall_start;

    if (rank == 0) {
        approx = 0.0;
        for (int r = 0; r < size; r++) {
            approx += all_partial[r];
        }
        err_abs = fabs(approx - exact);


        printf("M,threads,N,wall_s,kernel_s_max,approx,error,pass\n");
        printf("%d,%d,%llu,%.9f,%.9f,%.15e,%.6e,%s\n", size, threads_per_block,
               n_intervals, wall_elapsed, max_kernel_s, approx, err_abs,
               (err_abs <= 1e-8) ? "yes" : "no");
        printf("exact=%.15e\n", exact);
        printf("note=run multiple M and -t values to build a 3D runtime surface\n");
    }
```

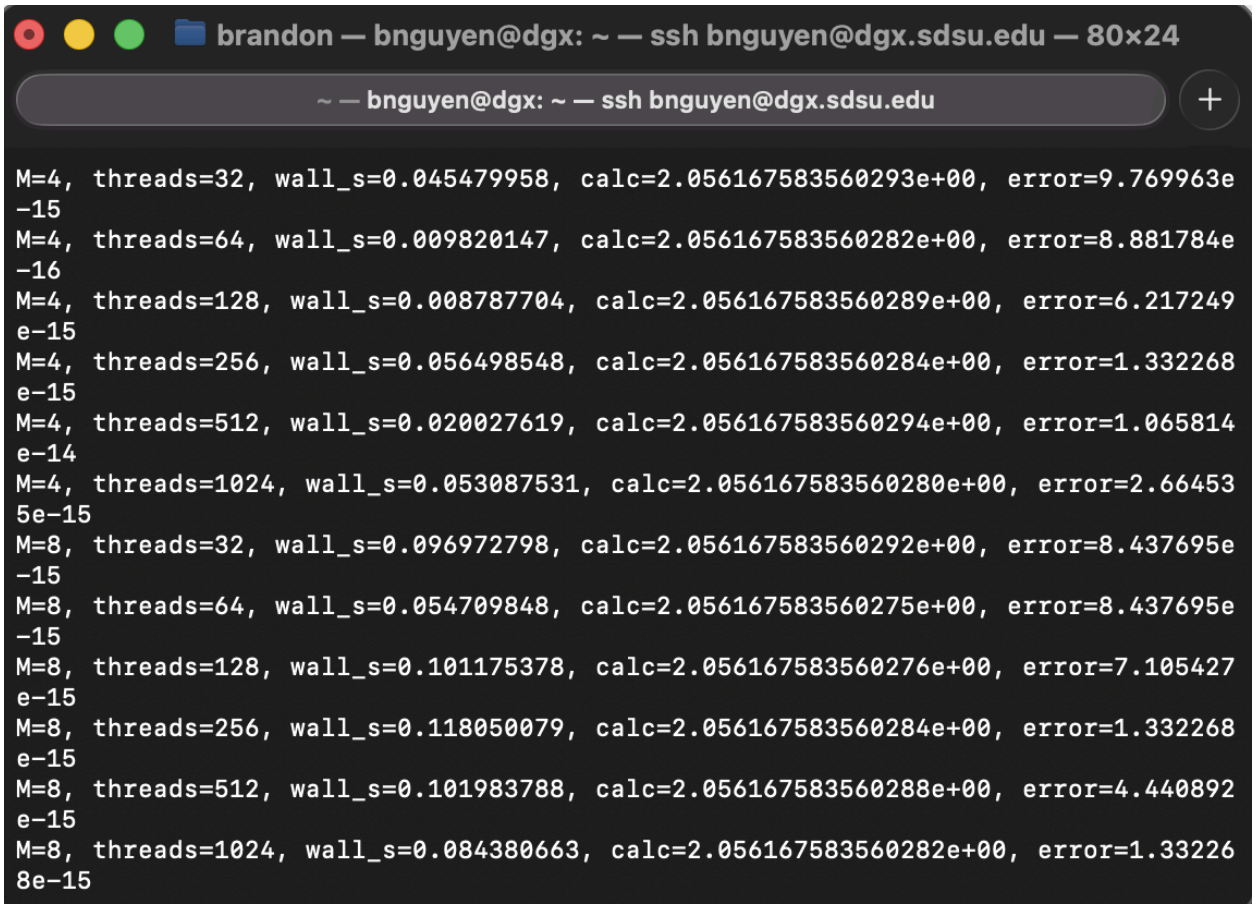
```
    if (all_partial != NULL) {
        free(all_partial);
    }
    if (h_block_sums != NULL) {
        free(h_block_sums);
    }
    if (d_block_sums != NULL) {
        CUDA_CHECK(cudaFree(d_block_sums));
    }
    CUDA_CHECK(cudaEventDestroy(ev_start));
    CUDA_CHECK(cudaEventDestroy(ev_stop));

    MPI_Finalize();
    return 0;
}
```

// Makefile

```
NVCC         := nvcc
MPICXX       := mpicxx
TARGET       := simpson_mpi_cuda
SRC          := simpson_mpi_cuda.cu
N            ?= 10000000
M            ?= 1
T            ?= 256
M_VALUES     ?= 2 4 8
THREAD_VALUES ?= 32 64 128 256 512 1024 2048
CSV_OUT      ?= p11_surface.csv


SHELL := /bin/bash

.PHONY: all build run sweep clean help

all: build

help:
    @echo "Targets:"
    @echo "  make build
    @echo "  make run M=4 N=10000000 T=256
    @echo "  make sweep N=10000000
```

```
    @echo ""
    @echo "Cluster reminder:"
    @echo "  module load openmpi/4.1.7"
    @echo "  (and load CUDA module if needed)"

build: $(TARGET)

$(TARGET): $(SRC)
    $(NVCC) -x cu -ccbin $(MPICXX) -O3 -o $(TARGET) $(SRC) -lm

run: build
    mpirun -mca btl "^openib" -np $(M) ./$(TARGET) -n $(N) -t $(T)

sweep: build
    @echo "M,threads,N,wall_s,kernel_s_max,approx,error,pass" > $(CSV_OUT)
    @echo "Sweep stats (M, threads, wall_s, calculation, error):"
    @for m in $(M_VALUES); do \
        for t in $(THREAD_VALUES); do \
            out="$$(mpirun -mca btl "^openib" -np $$m ./$(TARGET) -n $(N) -t $$t)"; \
            row="$$(echo "$$out" | awk -F, 'BEGIN{OFS=","} /^[0-9]+,[0-9]+,[0-9]+,/ {print
$$0; exit}')"; \
            if [ -n "$$row" ]; then \
                echo "$$row" >> $(CSV_OUT); \
                echo "$$row" | awk -F, '{printf "M=%s, threads=%s, wall_s=%s, calc=%s,
error=%s\n", $$1, $$2, $$4, $$6, $$7}'; \
            else \
                echo "Run failed for M=$$m T=$$t"; \
                echo "$$out"; \
                exit 1; \
            fi; \
        done; \
    done
    @echo "Wrote sweep data to $(CSV_OUT)"

clean:
    rm -f $(TARGET) $(CSV_OUT)
```

4) Terminal Output



5) Discussion

In P11, we are tasked with modifying our composite 1/3 Simpson's quadrature method code from P03 to perform the integration over M MPI processes, where each process (e.g., MPI rank) integrates over a unique range of N/M intervals, where N is the total number of intervals specified on the command line as an argument. After modifying the program using a combination of MPI and CUDA, we can run the program to compare. For this assignment, we will be using M = 2^1 to 2^3 ranks with a CUDA thread count as a power of 2 from 2^0 to 2^11 threads, and generate a 3D runtime (wall clock) surface plot as a function of M and thread count for a specific value of N as a power of 10. According to my findings, we can see that rank M = 4 is the optimal value in minimizing runtime because it has the lowest points between threads 64 and 128 for my comparison graph. With that being said, this final programming assignment for the course shows us how a hybrid implementation works and continues to expand our knowledge on CUDA programming.